

# Rapport de Projet : CookMind

Application de génération de recettes avec Next.js et Supabase

Boukhari Melina & Meriem Fekih

17 mai 2026

## Table des matières

|          |                                                                   |          |
|----------|-------------------------------------------------------------------|----------|
| <b>1</b> | <b>Présentation du projet</b>                                     | <b>3</b> |
| <b>2</b> | <b>Choix technologiques</b>                                       | <b>3</b> |
| <b>3</b> | <b>Installation et Configuration du Projet</b>                    | <b>3</b> |
| 3.1      | Installation des paquets . . . . .                                | 3        |
| 3.2      | Configuration des variables d'environnement . . . . .             | 3        |
| 3.3      | Lancement du projet . . . . .                                     | 4        |
| <b>4</b> | <b>Structure et fonctionnement du Front-End (Next.js / React)</b> | <b>4</b> |
| 4.1      | Différence entre Server et Client Components . . . . .            | 4        |
| 4.2      | Gestion d'une erreur d'affichage React . . . . .                  | 4        |
| <b>5</b> | <b>Logique Serveur et Base de données</b>                         | <b>4</b> |
| 5.1      | Les Server Actions . . . . .                                      | 4        |
| 5.2      | Structure de la base de données Supabase . . . . .                | 4        |
| <b>6</b> | <b>Conclusion</b>                                                 | <b>5</b> |



### 3.3 Lancement du projet

Le serveur de développement local est ensuite démarré via la commande suivante, rendant l'application accessible sur `http://localhost:3000` :

```
npm run dev
```

## 4 Structure et fonctionnement du Front-End (Next.js / React)

Next.js 15 fonctionne par défaut avec des *Server Components*. J'ai dû diviser ma logique en deux parties pour que l'application fonctionne correctement :

### 4.1 Différence entre Server et Client Components

- **Server Components** (`page.tsx`) : C'est la page principale. Elle s'exécute côté serveur pour récupérer de manière sécurisée les anciennes recettes de l'utilisateur dans Supabase, sans exposer de clés privées.
- **Client Components** (`content.tsx`) : J'ai ajouté la directive `"use client"` en haut de ce fichier car il gère l'interactivité. Il contient les états React (`useState`) pour stocker la saisie du formulaire, le chargement du bouton, et l'affichage dynamique du résultat de l'IA.

### 4.2 Gestion d'une erreur d'affichage React

Pendant le développement, j'ai rencontré l'erreur : *"Objects are not valid as a React child"*. Cela venait du fait que les ingrédients de la recette ou de l'historique arrivaient sous forme d'objets JSON complexes `{name, quantity, unit}` depuis la base de données. React ne peut pas afficher un objet brut directement dans le texte.

Pour corriger ce bug, j'ai modifié le code pour vérifier le type de la variable avant le rendu :

```
{recipe.recipe?.ingredients_list?.map((ing, i) => (  
  <Row  
    key={i}  
    label={typeof ing === 'object' ? `${ing.quantity} ${ing.unit} ${ing.name}` : ing}  
    value=""  
  />  
)})}
```

## 5 Logique Serveur et Base de données

### 5.1 Les Server Actions

Au lieu de créer des routes API classiques, le projet utilise les *Server Actions* de Next.js (`"use server"`). Quand l'utilisateur clique sur le bouton, la fonction appelle directement l'API d'OpenAI côté serveur (ce qui protège ma clé API), récupère le JSON, puis exécute les insertions dans Supabase.

### 5.2 Structure de la base de données Supabase

Une fois la recette générée, le script effectue trois insertions d'affilée dans la base de données à l'aide de l'ID de la recette :

1. **Table recipes** : Stocke le titre, le type de plat, le temps de préparation (`prep_time_min`) et de cuisson (`cook_time_min`).

2. **Table nutrition\_analyses** : Enregistre les calories, les macros (protéines, glucides, lipides) et les micros (fibres, vitamines, minéraux). Les valeurs sont converties en nombres via `parseFloat` pour correspondre au type `numeric` de Supabase.
3. **Table shopping\_lists** : télécharge la liste de courses générée au format PDF.

## 6 Conclusion

Le projet est entièrement fonctionnel. L'utilisation conjointe de Next.js et de Supabase permet de lier efficacement une IA générative avec une base de données relationnelle, tout en ayant une interface réactive et sécurisée.